NUMMERISCHE VERFAHREN ZUR NULLSTELLENBESTIMMUNG

Luca Wendt 3BHWII

30. APRIL 2026

Aufgabe 5 – Bisektionsverfahren

In dieser Aufgabe wurde das Bisektionsverfahren zur Bestimmung von Nullstellen verwendet.
Dabei wird ein Intervall a, b gewählt, in dem ein Vorzeichenwechsel vorliegt, also:

$$f(a) * f(b) < 0$$

In jedem Schritt wird das Intervall halbiert und der Mittelpunkt c berechnet. Anschließend wird überprüft, in welchem Teilintervall der Vorzeichenwechsel liegt. Dieser Prozess wird wiederholt, bis der Funktionswert f(c) nahe genug bei 0 ist.

Code Erklärung:

```python
import math
from typing import Tuple

def f(x: float, funktion: str) -> float:
    """Berechnet den Funktionswert für eine gegebene Funktion und Variable x

    Args:
        x (float): Funktionsvariable
        funktion (str): Funktion als string

    Raises:
        ValueError: Bei Fehler in der Funktion wird ein ValueError mit einer Fehlermeldung ausgelöst

    Returns:
        float: Funktionswert wird als float zurückgegeben
    """

    try:
        return eval(funktion)       #eval macht aus dem String eine ausführbare Fuktion, Variable x wird in der Funktion verwendet
    except Exception as fehler:
        raise ValueError(f"Fehler in der Funktion: {fehler}")
```

Im Code wird zuerst die Funktion f(x, Funktion) verwendet. Sie berechnet den Funktionswert für einen bestimmten x- Wert. Die Funktion selbst wird aber als String eingegeben und mit **eval()** ausgewertet.

```python
def bisektion(funktion: str, a: float, b: float, epsilon:float = 0.00001) -> Tuple[float, int]:
    """Nullstelle wird anhand von bisektion gesucht.
    Abbruch, wenn f(c) nahe genug bei 0 ist.

    Args:
        funktion (str): Funktionswert
        a (float): 1. Intervallsgrenze
        b (float): 2. Intervallsgrenze
        epsilon (float, optional): Genauigkeit

    Raises:
        ValueError: Kein gültiges Startintervall
        ValueError: Fehler im Intervall

    Returns:
        Tuple[float, int]: Näherung der Nullstelle, Anzahl der Iterationen
    """


    fa = f(a, funktion)
    fb = f(b, funktion)

    try:
        if fa * fb > 0:
            raise ValueError("Kein gültiges Startintervall")

        iteration = 0

        while True:
            c = (a + b) / 2
            fc = f(c, funktion)
            iteration += 1

            if abs(fc) <= epsilon:
                return c, iteration

            if fa * fc < 0:
                b = c
                fb = fc

            elif fb * fc < 0:
                a = c
                fa = fc

            else:
                raise ValueError("Fehler im Intervall")
    except ValueError as fehler:
        print(f"Fehler: {fehler}")
        return 0, 0
```

Danach wird die Funktion bisektion() erstellt, die nach der Nullstelle sucht. Dabei werden die Funktionswerte an den Intervallgrenzen a und b berechnet. Wenn kein Vorzeichen vorhanden ist, wird eine Fehlermeldung ausgeben. Danach wird in einer Schleife immer der Mittelpunkt c berechnet. Wenn f(c) nahe genug bei 0 liegt wird c als Näherung zurückgegeben. Sonst wird das Intervall angepasst.

```python
def wurzel_bisektiont_solver(n):
    # Verwendet zum Testen und vergleichen der numerischen Näherung mit der analytischen Lösung der Quadratwurzel von n

    funktion = f"x**2 - {n}"

    a = 0
    b = 2 * n

    näherung, iterationen = bisektion(funktion, a, b)
    analytisch = math.sqrt(n)

    print("-" * 50)
    print(f"n =     {n}")
    print(f"Näherung:       {näherung}")
    print(f"Exakter Punkt:      {analytisch}")
    print(f"Fehler:     {abs(näherung - analytisch)}")
    print(f"Iterationen:        {iterationen}")
    print("-" * 50)
```

Danach kommt ein Solver zum Berechnen der Nummerischen Näherung im Vergleich zu der analytischen Lösung (Quadratwurzel).
Dabei wird die Funktion auf f(x^2 – n) gesetzt, n ist hier eine Variable, die man beim Aufrufen der Funktion angibt. Dann werden die Werte a und b definiert und anschließend die Näherung und Iterationen der Bisektion von der Funktion und a, b gespeichert. Im Anschluss wird die Wurzel als Variable gespeichert und alles geprinted.

```python
def solver(funktion: str = None, a: float = None, b: float = None, epsilon: float = None) -> Tuple[float, int]:
    """Führt das Bisektionsverfahren aus, es kann entweder mit eingabe oder mit übergebenen werten arbeiten.

    Args:
        funktion (str, optional): funktion als string Default as None
        a (float, optional): 1. Intervallsgrenze. Defaults to None.
        b (float, optional): 2. Intervallsgrenze. Defaults to None.
        epsilon (float, optional): Genauigkeit. Defaults to None.

    Returns:
        Tuple[float, int]: Näherung der Nullstelle, Anzahl der Iterationen
    """


    try:
        if funktion is None:
            funktion = input("Gib die Funktion ein: ")
            a = float(input("Gib die erste Intervallsgrenze ein: "))
            b = float(input("Gib die zweite Intervallsgrenze ein: "))
            epsilon = float(input("Gib die Genauigkeit ein: "))

        näherung, iterationen = bisektion(funktion, a, b, epsilon)

        print("-" * 50)
        print(f"Funktion: f(x) = {funktion}")
        print(f"Näherung: {näherung}")
        print(f"Iterationen: {iterationen}")
        print("-" * 50)

        return näherung, iterationen

    except ValueError as fehler:
        print(f"Fehler: {fehler}")
        return 0,0
```

Dann wird noch eine Allgemeine Funktion zum Lösen gebildet. Diese Funktion überprüft zuerst, ob ein Wert gegeben wurde, falls nicht werden die Werte per input abgefragt. Danach wird die Näherung der Nullstelle und Anzahl der Iterationen ausgegeben. Durch das try/ except können ungültige Eingaben abgefangen werden.

Da die Aufgabe verlangte, dass man die Werte n = 25, 81, 144 prüfen soll ist das das Resultat:

```
--------------------------------------------------
n =     25
Näherung:       4.999999701976776
Exakter Punkt:      5.0
Fehler:     2.980232238769531e-07
Iterationen:        25
--------------------------------------------------
--------------------------------------------------
n =     81
Näherung:       8.999999463558197
Exakter Punkt:      9.0
Fehler:     5.364418029785156e-07
Iterationen:        25
--------------------------------------------------
--------------------------------------------------
n =     144
Näherung:       12.000000357627869
Exakter Punkt:      12.0
Fehler:     3.5762786865234375e-07
Iterationen:        28
--------------------------------------------------
```

Aufgabe 6 – Regula Falsi

In dieser Aufgabe wird dasselbe gefragt wie bei Aufgabe 5.
Ich habe mich für die Methode des Regula falsi entschieden. Diese Unterscheidet sich nur in der Berechnung von c. Bei Regula Falsi wird c nicht als Mittelpunkt berechnet, sondern mit der Sekantenformel.
Daher das der Gesamte Code, dem der Aufgabe 5 sehr ähnelt, wird der Code wird der Unterschied erklärt.
Code Erklärung:

```python
def regula_falsi(funktion: str, a: float, b: float, epsilon:float = 0.00001) -> Tuple[float, int]:
    """Nullstelle wird anhand von regula falsi gesucht.
    Abbruch, wenn f(c) nahe genug bei 0 ist.

    Args:
        funktion (str): Funktionswert
        a (float): 1. Intervallsgrenze
        b (float): 2. Intervallsgrenze
        epsilon (float, optional): Genauigkeit

    Raises:
        ValueError: Kein gültiges Startintervall
        ValueError: Fehler im Intervall

    Returns:
        Tuple[float, int]: Näherung der Nullstelle, Anzahl der Iterationen
    """


    fa = f(a, funktion)
    fb = f(b, funktion)

    try:
        if fa * fb > 0:
            raise ValueError("Kein gültiges Startintervall")

        iteration = 0

        while True:
            c = b - fb * (b - a)/(fb - fa)
            fc = f(c, funktion)
            iteration += 1

            if abs(fc) <= epsilon:
                return c, iteration

            if fa * fc < 0:
                b = c
                fb = fc

            elif fb * fc < 0:
                a = c
                fa = fc

            else:
                raise ValueError("Fehler im Intervall")
    except ValueError as fehler:
        print(f"Fehler: {fehler}")
        return 0, 0
```

Der Code unterscheidet sich bei der Berechnung von c, hierbei wird stattdessen gerechnet: c = b – fb * (b- a) / (fb – fa)

Und da diese Aufgabe wieder einen Test der Ergebnisse mit dem Zahlenwert von Aufgabe 1,2,5 ist das das Resultat:

```
--------------------------------------------------
Aufgabe 1:
n =     67
Näherung:       8.185352512320081
Exakter Punkt:      8.18535277187245
Fehler:     2.5955236893082656e-07
Iterationen:        659
--------------------------------------------------
Aufgabe 2:
n =     19
Näherung:       4.358897809317309
Exakter Punkt:      4.358898943540674
Fehler:     1.1342233650069034e-06
Iterationen:        243
--------------------------------------------------
Aufgabe 5.1:
n =     25
Näherung:       5.000000032788943
Exakter Punkt:      5.0
Fehler:     3.278894311620206e-08
Iterationen:        353
--------------------------------------------------
Aufgabe 5.2:
n =     81
Näherung:       8.999999604016649
Exakter Punkt:      9.0
Fehler:     3.959833509981081e-07
Iterationen:        487
--------------------------------------------------
Aufgabe 5.3:
n =     144
Näherung:       11.999999918781167
Exakter Punkt:      12.0
Fehler:     8.121883254830209e-08
Iterationen:        553
--------------------------------------------------
```

Aufgabe 7 – Visualisierung

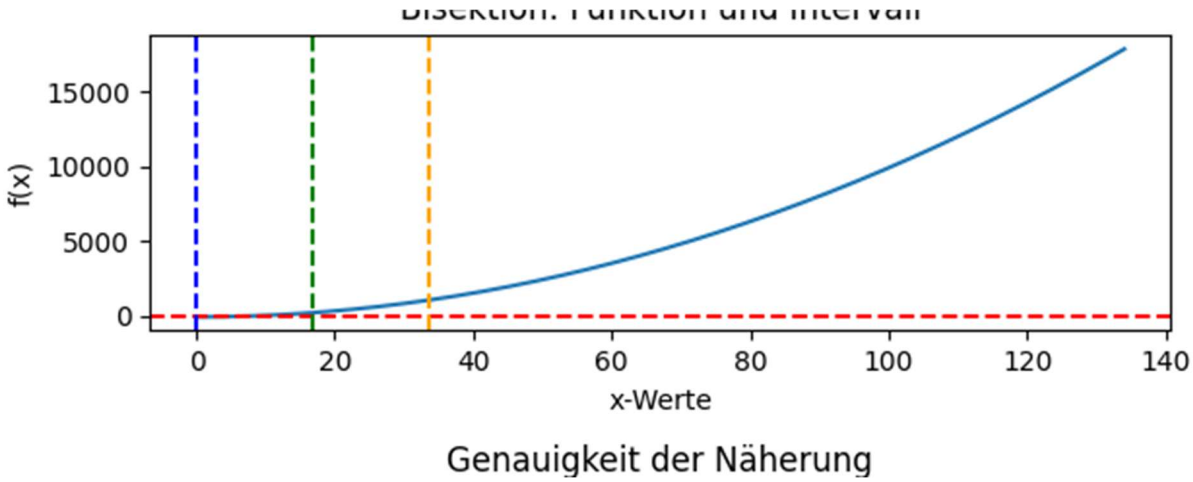
In Aufgabe 7 wird verlangt, dass man die Intervallshalbierung visualisiert.

```python
def bisektion_iterationen(funktion: str, a: float, b: float, epsilon: float = 0.000001) -> list:
    """Führt das Bisektionsverfahren aus und speichert alle Iterationen

    Args:
        funktion (str): Funktion als string
        a (float): 1. Intervallsgrenze
        b (float): 2. Intervallsgrenze
        epsilon (float, optional): Genauigkeit

    Returns:
        list: Liste mit allen Iterationen (a, b, c, fc)
    """

    iterationen = []

    while True:
        c = (a + b) / 2
        fc = f(c, funktion)

        #speichert aktuelles Intervall und Näherung der Nullstelle
        iterationen.append((a, b, c, fc))

        #Abbruchbedingung
        if abs(fc) <= epsilon:
            break

        #Intervall wird halbiert (Vorzeichenwechsel prüfen)
        if f(a, funktion) * fc < 0:
            b = c
        else:
            a = c

    return iterationen
```

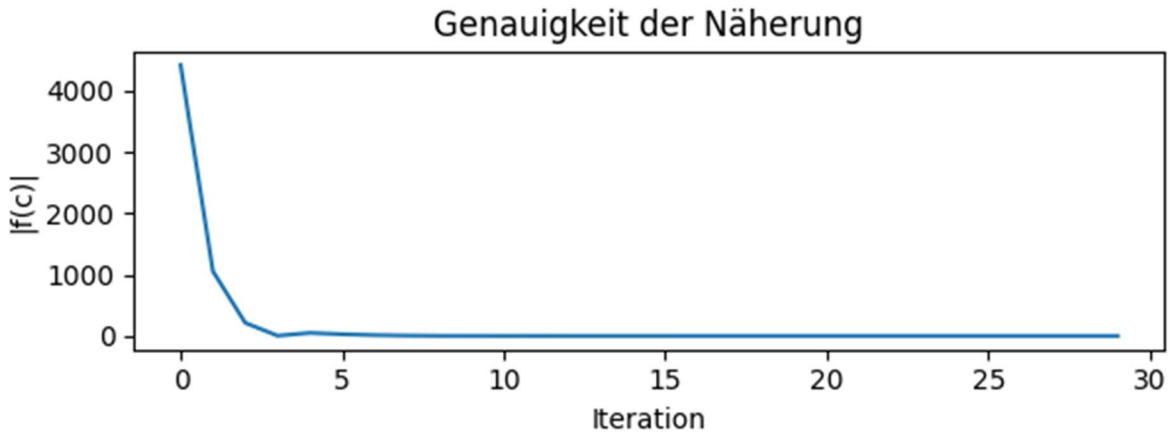
Hier werden die Iterationen der Bisektion gespeichert. Dafür gibt es eine Liste Iterationen. In jeder Iteration werden die Werte: a, b, c und f(c) gespeichert.

```python
def plotter(iterationen: list, funktion: str, a: float, b: float) -> None:

    Args:
        iterationen (list): Liste mit allen Iterationen (a, b, c, fc)
        funktion (str): Funktion als string
        a (float): 1. Intervallsgrenze
        b (float): 2. Intervallsgrenze
    """



    lösung = []
    genauigkeit = []

    #Erzeugt x-Werte für den Funktionsgraphen
    x = np.linspace(a, b, 500)
    y = [f(xi, funktion) for xi in x]

    plt.ion()

    for a, b, c, fc in iterationen:
        lösung.append(c)
        genauigkeit.append(abs(fc))

        plt.clf()

        #Diagramm 1: Funktion und Intervall
        plt.subplot(2, 1, 1)
        plt.plot(x,y)
        plt.axhline(0, color='red', linestyle='--')
        plt.axvline(c, color='green', linestyle='--')
        plt.axvline(a, color='blue', linestyle='--')
        plt.axvline(b, color='orange', linestyle='--')
        plt.title("Bisektion: Funktion und Intervall")
        plt.xlabel("x-Werte")
        plt.ylabel("f(x)")

        #Diagramm 2: Genauigkeit der Näherung
        plt.subplot(2, 1, 2)
        plt.plot(genauigkeit)
        plt.title("Genauigkeit der Näherung")
        plt.xlabel("Iteration")
        plt.ylabel("|f(c)|")


        plt.tight_layout()
        plt.pause(0.5)

    plt.ioff()
    plt.show()
```

Die Funktion plotter verwendet diese Liste, um die Berechnung das Diagramm darzustellen.



Im ersten Diagramm sieht man den Funktionsgraphen, das aktuelle Intervall und die aktuelle Näherung c.



Im 2. Diagramm wird die Genauigkeit f(c) angezeigt. Durch das plt.pause entsteht eine Animation

```
def solver() -> None:
    """Führt die berechnung und Visualisierung, mit gewünschten zahlwerten
    """


    funktion = input("Gibt eine Funktion ein: ")
    a = float(input("Gib die erste Intervallsgrenze ein: "))
    b = float(input("Gib die zweite Intervallsgrenze ein: "))

    iterationen = bisektion_iterationen(funktion, a, b)
    plotter(iterationen, funktion, a, b)
```

Bei dem Solver werden die Werte abgefragt und die Diagramme geplottet

Aufgabe 8 – Polynom

In Aufgabe 8 wird verlangt, dass man den Solver von Aufgabe 5 & 6 auf das Polynom anwendet.

```python
def solver_aufgabe8() -> None:
    """Testet das Bisektionsverfahren mit dem Polynom f(x) = 2*x + x**2 + 3*x**3 - x**4 im Intervall [3, 4] und verschiedenen Genauigkeiten

        Returns: None
    """

    funktion = "2*x + x**2 + 3*x**3 - x**4"
    a = 3
    b = 4


    print("-" * 50 )
    print("Genauigkeit: 0.01")
    solver(funktion, a, b, epsilon=0.01)
    print("-" * 50)

    print("Genauigkeit: 0.00000001")
    solver(funktion, a, b, epsilon=0.00000001)
    print("-" * 50)
```

Hier wird die Funktion definiert P(x) = 2x +x^2 +3x^3−x^4
Danach die Grenzen festgelegt, ich habe mich hier für 3 und 4 entschieden, weil sich die Nullstelle (3,4567) zwischen diesen Werten befindet. Im Anschluss wird das Polyonom mit verschiedenen Genauigkeiten abgefragt.

```
--------------------------------------------------
Genauigkeit: 0.01
--------------------------------------------------
Funktion: f(x) = 2*x + x**2 + 3*x**3 - x**4
Näherung: 3.45654296875
Iterationen: 11
--------------------------------------------------
--------------------------------------------------
Genauigkeit: 0.00000001
--------------------------------------------------
Funktion: f(x) = 2*x + x**2 + 3*x**3 - x**4
Näherung: 3.4566783430054784
Iterationen: 30
--------------------------------------------------
--------------------------------------------------
```

Für die Genauigkeit von 10^-2 werden 11 Iterationen gebraucht.

Bei einer Genauigkeit von 10^-8 werden 30 Iterationen gebraucht.

Aufgabe 9 – Seillänge

In Aufgabe 9 muss man mit dem Solver von Aufgabe 5&6 eine Seillänge berechnen.

Hierzu muss man die Gleichung der Kettenlinie: $y(x) = a * \cosh\left(\frac{x-x_0}{a}\right) - a + y_0$

$$y(50m) = y_0 + 10m$$

$$y(50m) = a \cdot \cosh\left(\frac{50}{a}\right) - a + y_0$$

$$y(50m) = y(50m)$$

$$a \cdot \cosh\left(\frac{50}{a}\right) - a + y_0 = y_0 + 10 \quad | -y_0$$

$$a \cdot \cosh\left(\frac{50}{a}\right) - a = 10 \quad | -10$$

$$a \cdot \cosh\left(\frac{50}{a}\right) - a - 10 = 0$$

Umformen, um ein Nullstellenproblem zu haben.

```python
def solver_aufgabe9() -> None:
    """Berechnet den Krümmungsradius einer Kettenlinie und die Länge des Seils, mithilfe vom Bisektionsverfahren und Regula Falsi-Verfahren (Überprüfung)

    Returns: None
    """


    print("-" * 50)
    print("Mit solver (Bisektionsverfahren):")
    funktion = "x * math.cosh(50/x) - x -10"


    a = 50
    b = 150
    epsilon = 0.00001

    radius, iterationen = solver(funktion, a, b, epsilon)

    w = 100

    länge = 2 * radius * math.sinh(w/(2*radius))

    print(f"Radius: {radius}")
    print(f"Anzahl der Iterationen: {iterationen}")
    print(f"Länge des Seils: {länge}")
    print("-" * 50)
    print("-" * 50)
    print("-" * 50)

    print("Mit solver2 (Regula Falsi):")

    radius, iterationen = solver2(funktion, a, b, epsilon)

    länge = 2 * radius * math.sinh(w/(2*radius))

    print(f"Radius: {radius}")
    print(f"Anzahl der Iterationen: {iterationen}")
    print(f"Länge des Seils: {länge}")
    print("-" * 50)
```

Da man die Funktion hat werden nun Intervallsgrenzen gesetzt. Ich habe 50 und 150 genommen. Danach werden Radius und Iterationen durch das Lösen der Funktion, a, b, Epsilon festgelegt. Mit diesen Werten kann man sich dann die Länge berechnen und am Ende wird alles geprinted.

Dasselbe habe ich dann noch mit der Regula Falsi Methode gemacht, um das Ergebnis 100% zu überprüfen und zu belegen.

```
--------------------------------------------------
Mit solver (Bisektionsverfahren):
--------------------------------------------------
Funktion: f(x) = x * math.cosh(50/x) - x -10
Näherung: 126.63249969482422
Iterationen: 19
--------------------------------------------------
Radius: 126.63249969482422
Anzahl der Iterationen: 19
Länge des Seils: 102.61868415967191
--------------------------------------------------
--------------------------------------------------
--------------------------------------------------
Mit solver2 (Regula Falsi):
--------------------------------------------------
Funktion: f(x) = x * math.cosh(50/x) - x -10
Näherung: 126.63251606976687
Iterationen: 28
--------------------------------------------------
Radius: 126.63251606976687
Anzahl der Iterationen: 28
Länge des Seils: 102.61868347714683
--------------------------------------------------
```